

Post-training & Reinforcement Learning III & Agents I

Aaron Mueller

CS599 B1: Advanced Natural Language Processing

Oct. 6, 2026

Dr. GRPO

Dr. GRPO

[Liu et al., 2025]

- GRPO:

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|\mathbf{o}_i|} \sum_{t=1}^{|\mathbf{o}_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] \right\}$$

$$\text{where } \hat{A}_{i,t} = \frac{r(\mathbf{q}, \mathbf{o}_i) - \text{mean}(\{r(\mathbf{q}, \mathbf{o}_1), \dots, r(\mathbf{q}, \mathbf{o}_G)\})}{\text{std}(\{r(\mathbf{q}, \mathbf{o}_1), \dots, r(\mathbf{q}, \mathbf{o}_G)\})}$$

- GRPO **D**one **r**ight (Dr. GRPO): just remove the std. dev. and length normalizations

$$\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|\mathbf{o}_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] \right\}$$

$$\text{where } \hat{A}_{i,t} = r(\mathbf{q}, \mathbf{o}_i) - \text{mean}(\{r(\mathbf{q}, \mathbf{o}_1), \dots, r(\mathbf{q}, \mathbf{o}_G)\})$$

Dr. GRPO

[Liu et al., 2025]

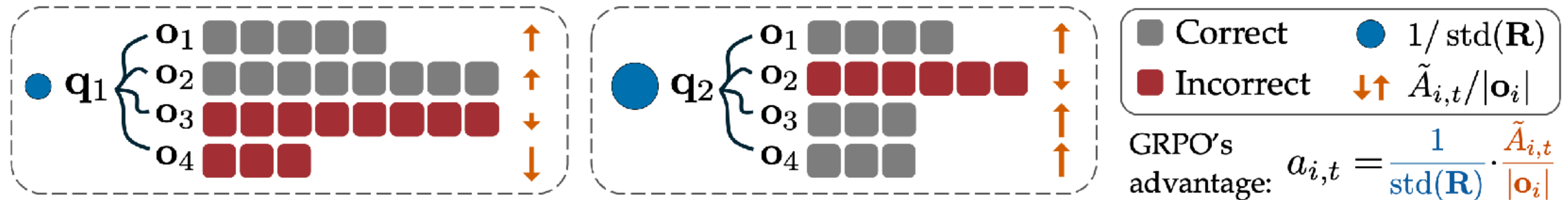
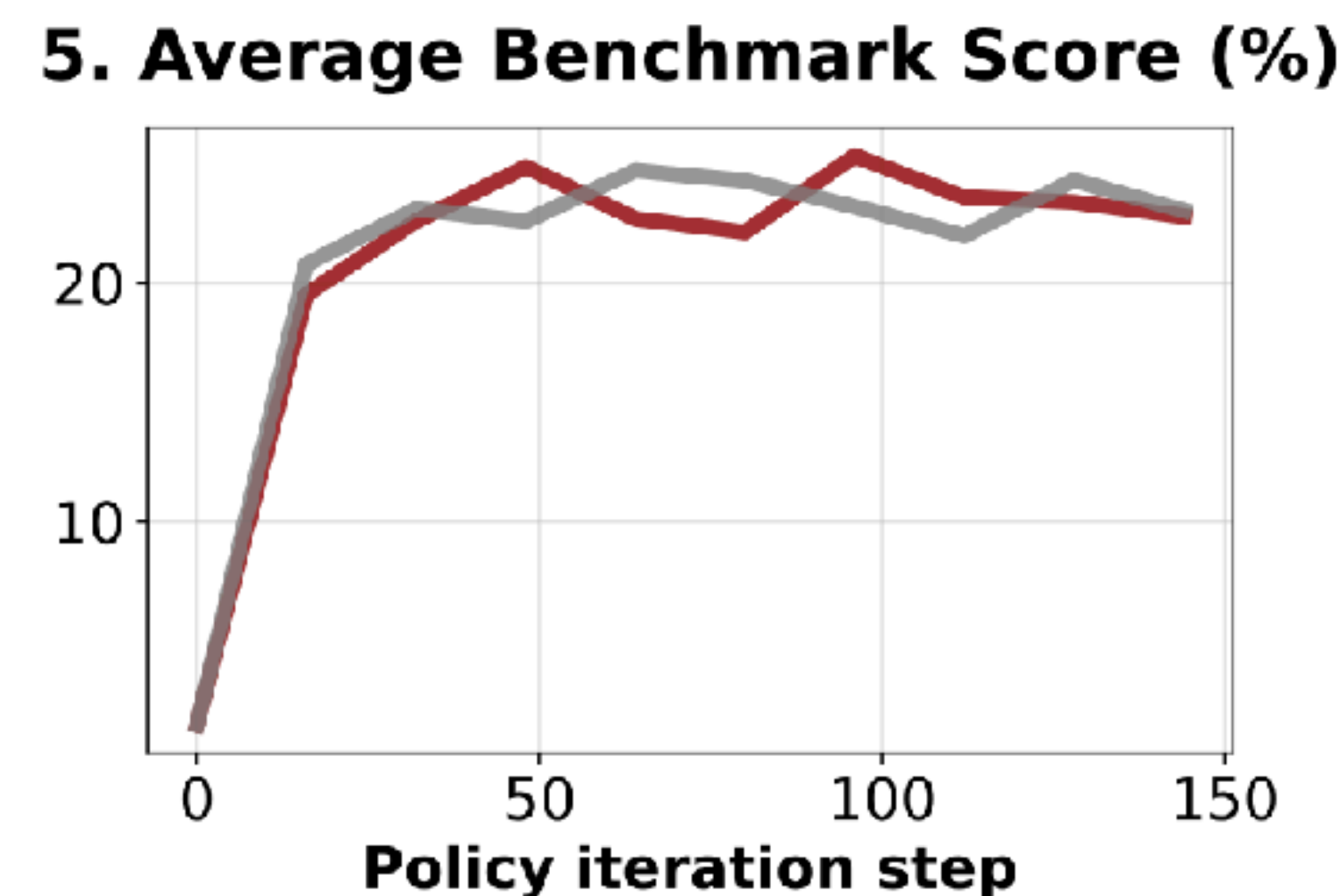
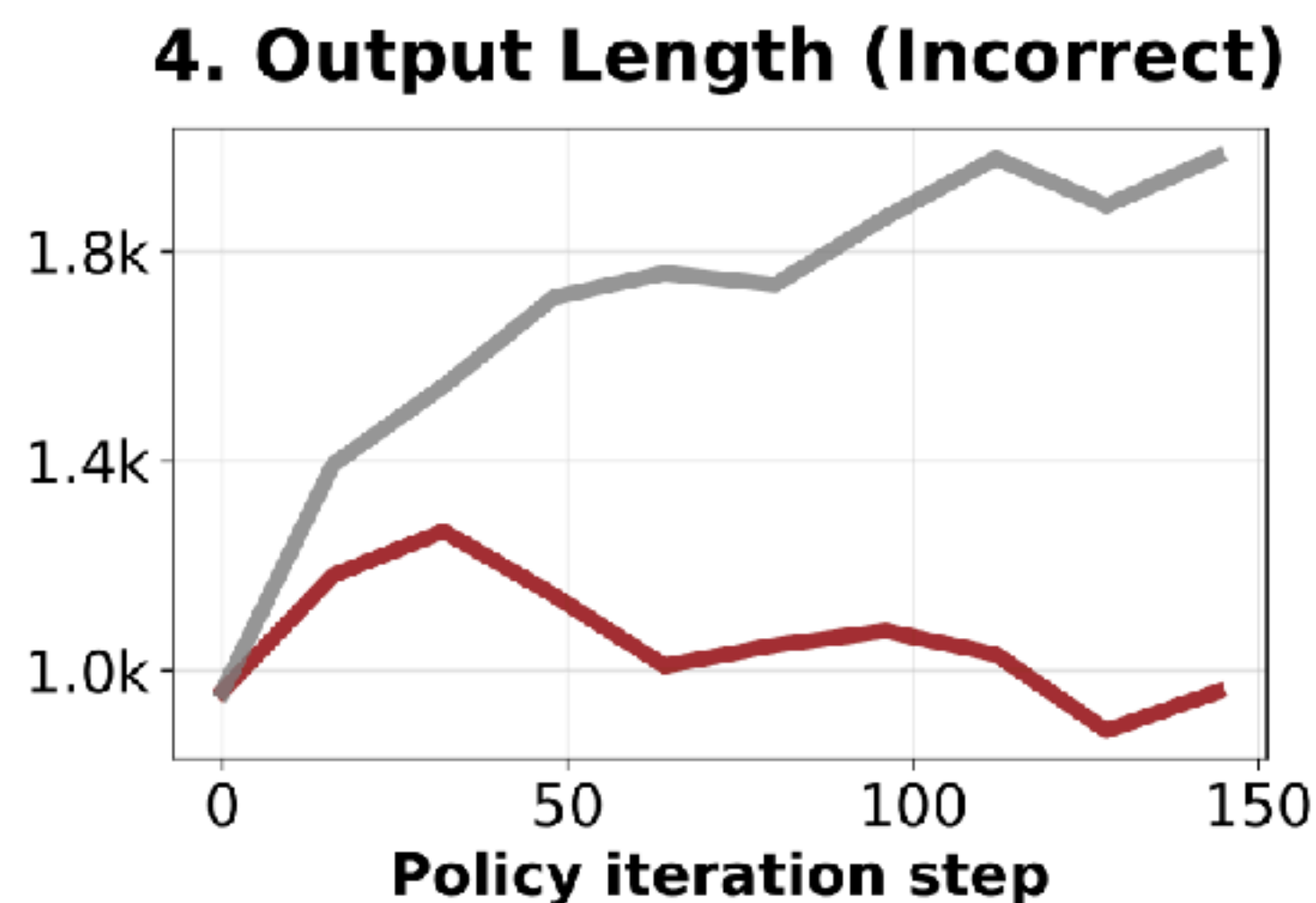
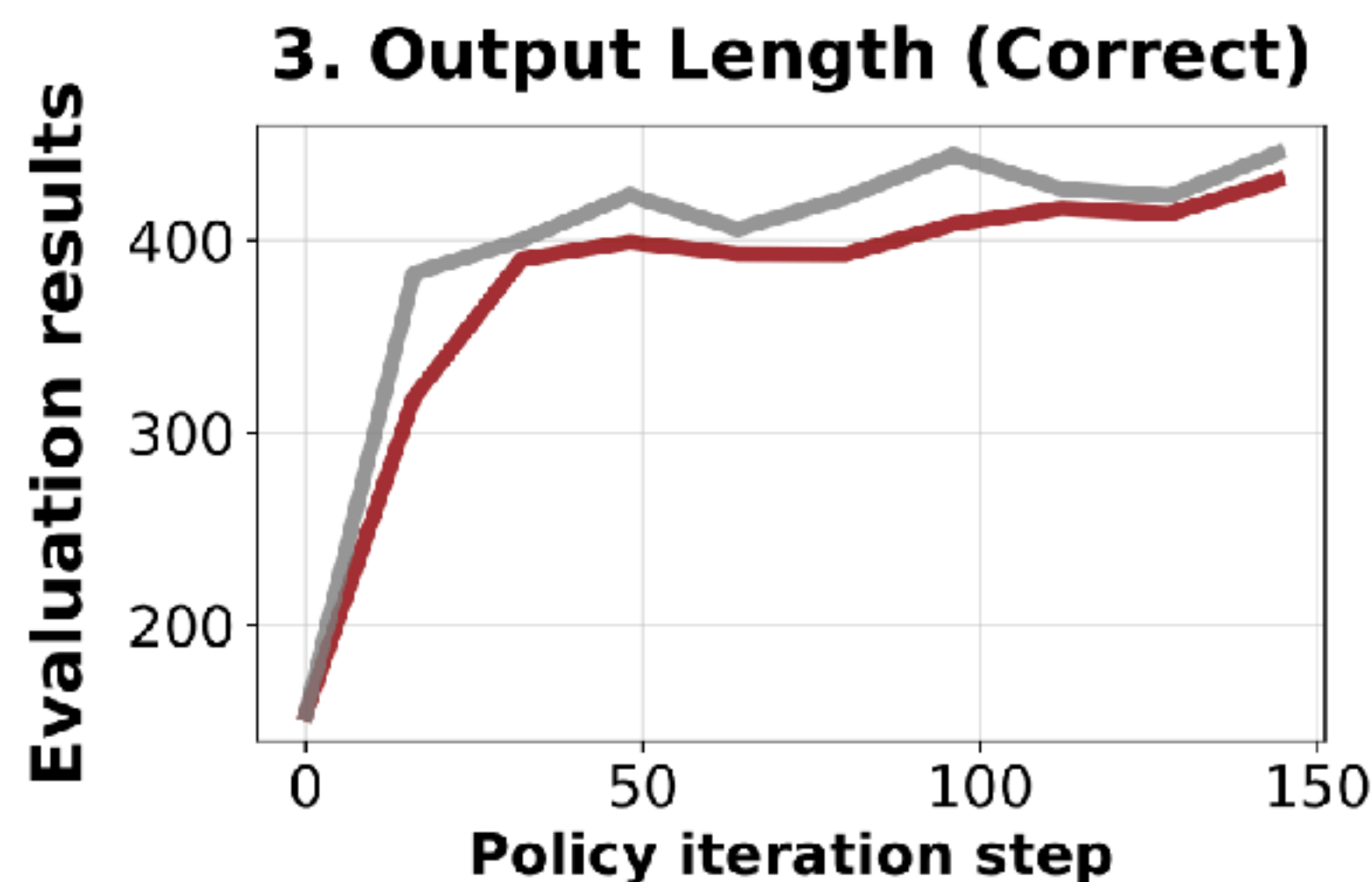
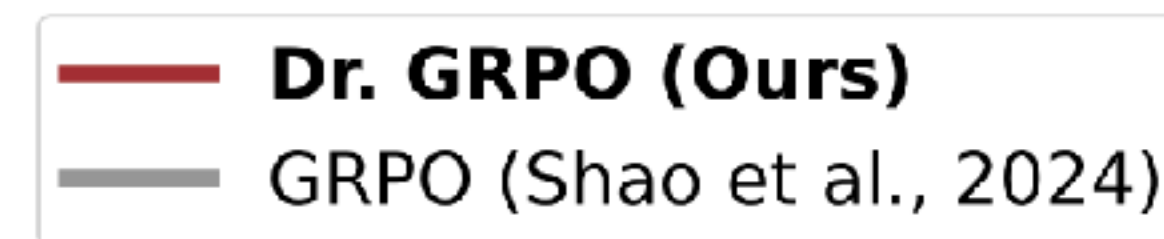
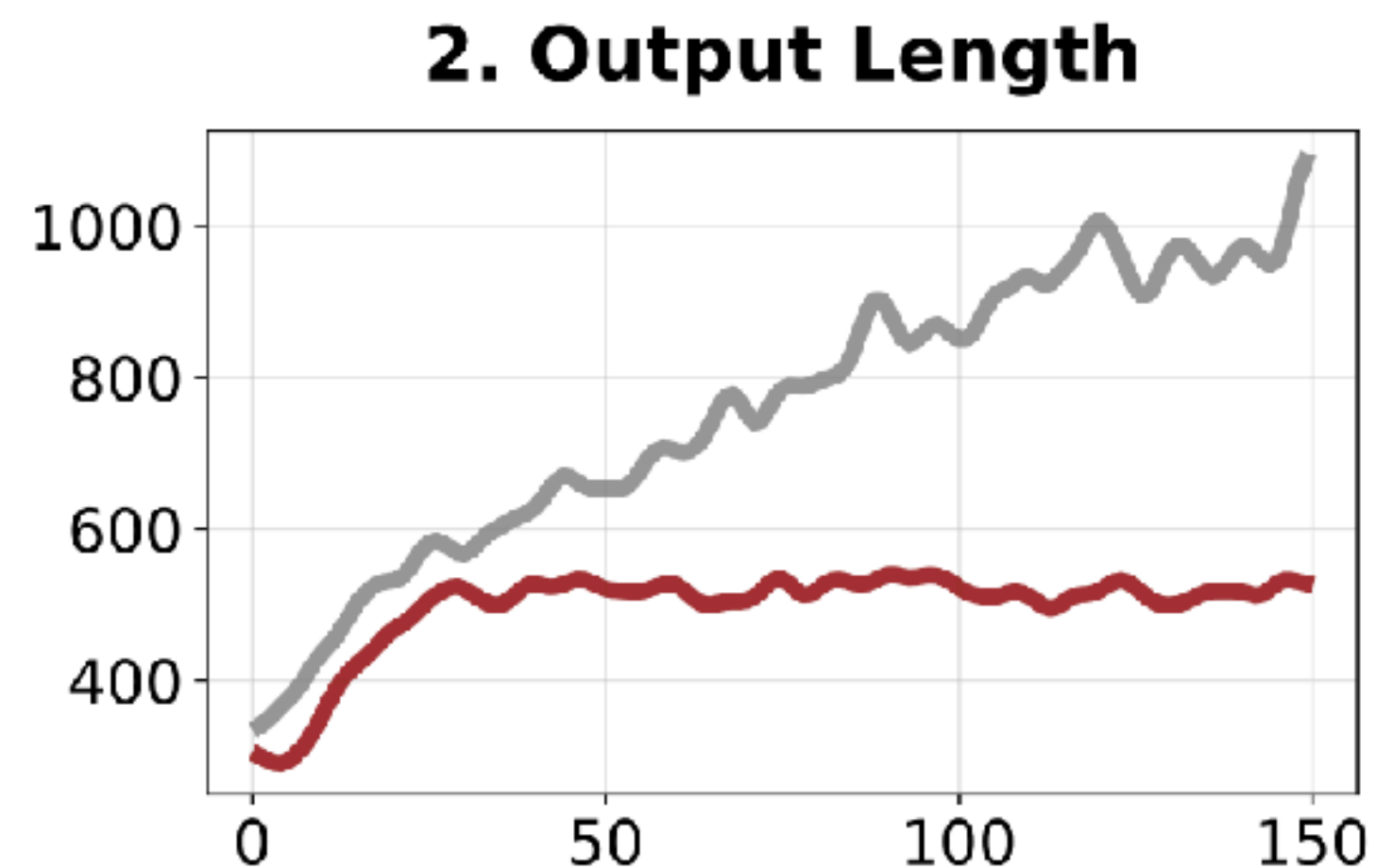
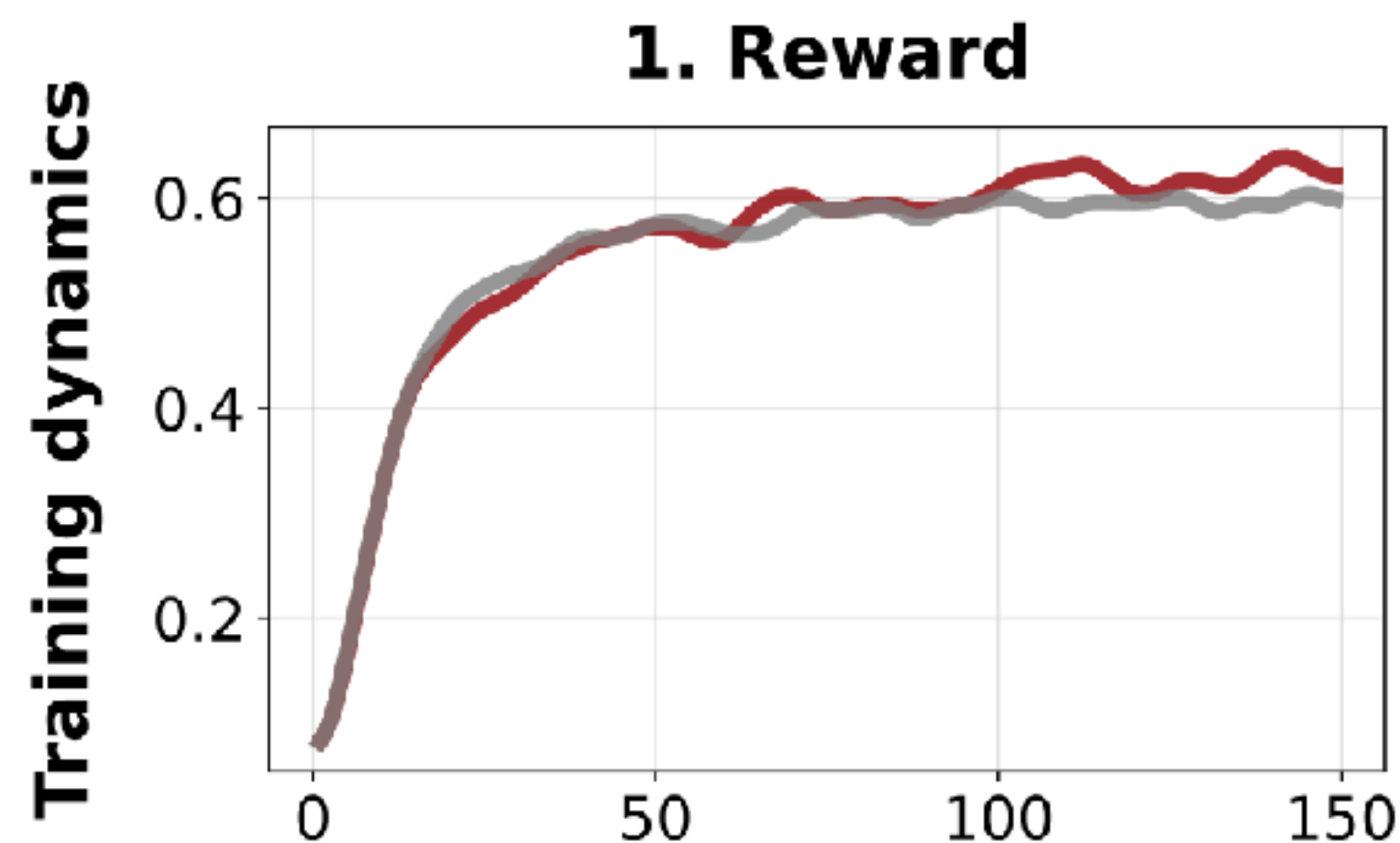


Figure 4: Illustration of the biases in GRPO. Note that the effective advantage of GRPO $a_{i,t}$ is equivalent to a reweighted version of the unbiased advantage $\tilde{A}_{i,t} = R(\mathbf{q}, \mathbf{o}_i) - \text{mean}(\mathbf{R})$. The terms $\text{std}(\mathbf{R})$ and $|\mathbf{o}_i|$ could bias the optimization by assigning different weights to different questions and responses, as denoted by the sizes of the blue circles and the lengths of the orange arrows. Upward arrows indicate positive advantages, and vice versa.

- With regular GRPO, shorter correct responses are boosted more, and longer incorrect responses are penalized less. Dr. GRPO fixes this
- Dr. GRPO also excludes the KL

Dr. GRPO

[Liu et al., 2025]



Does RL teach models new things, or elicit existing abilities?

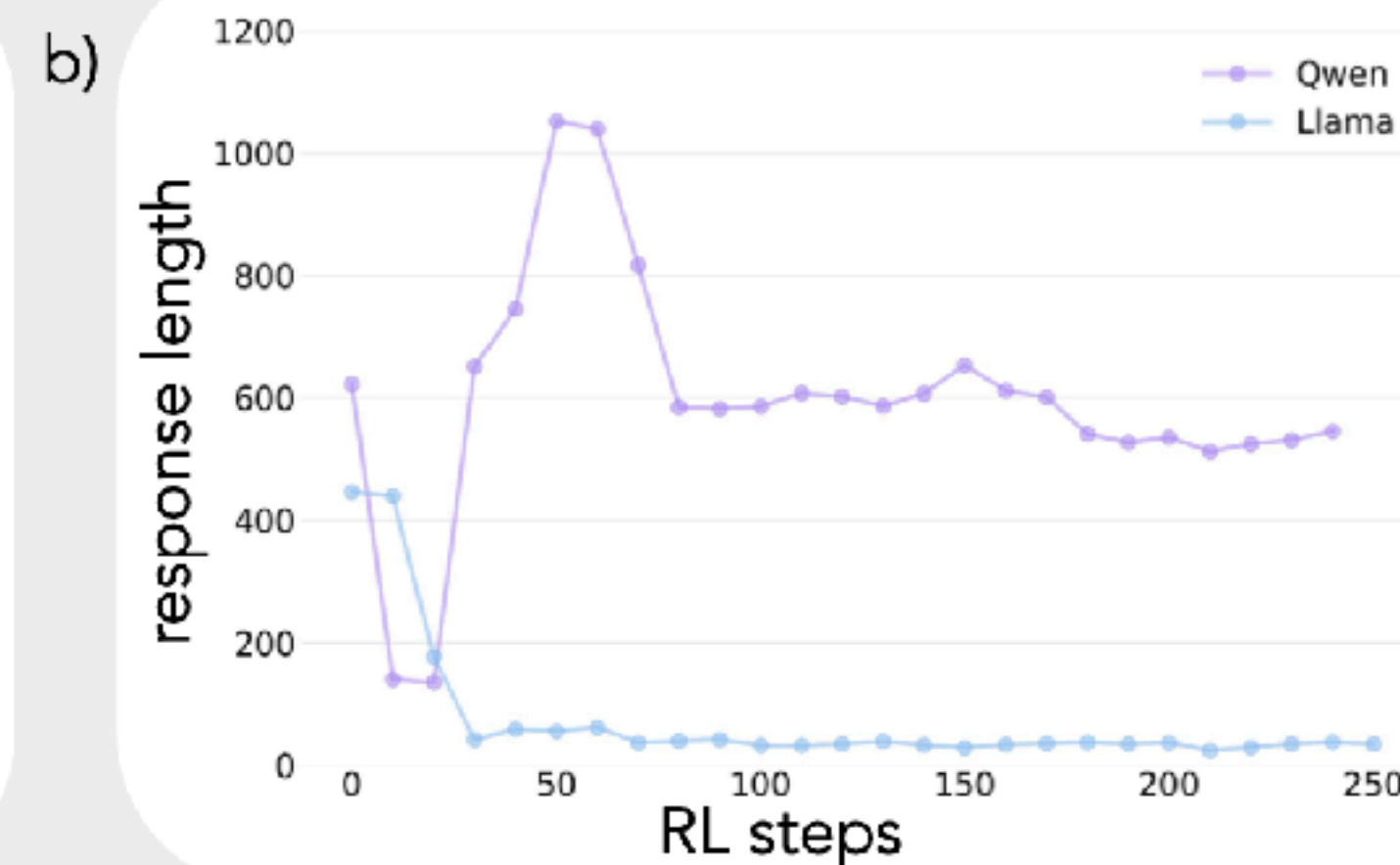
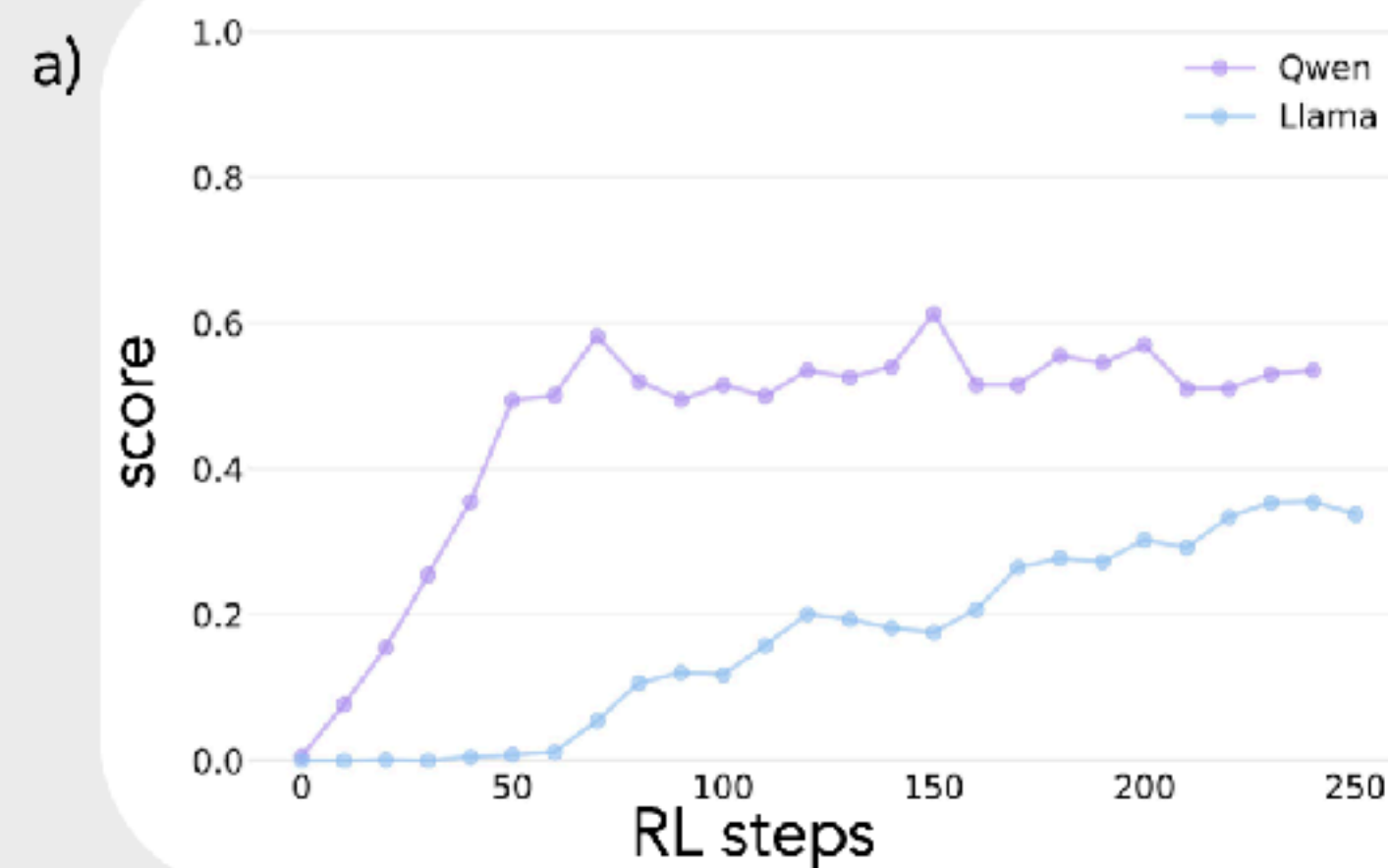
A tale of two models: Qwen 2.5 3B and Llama 3.2 3B



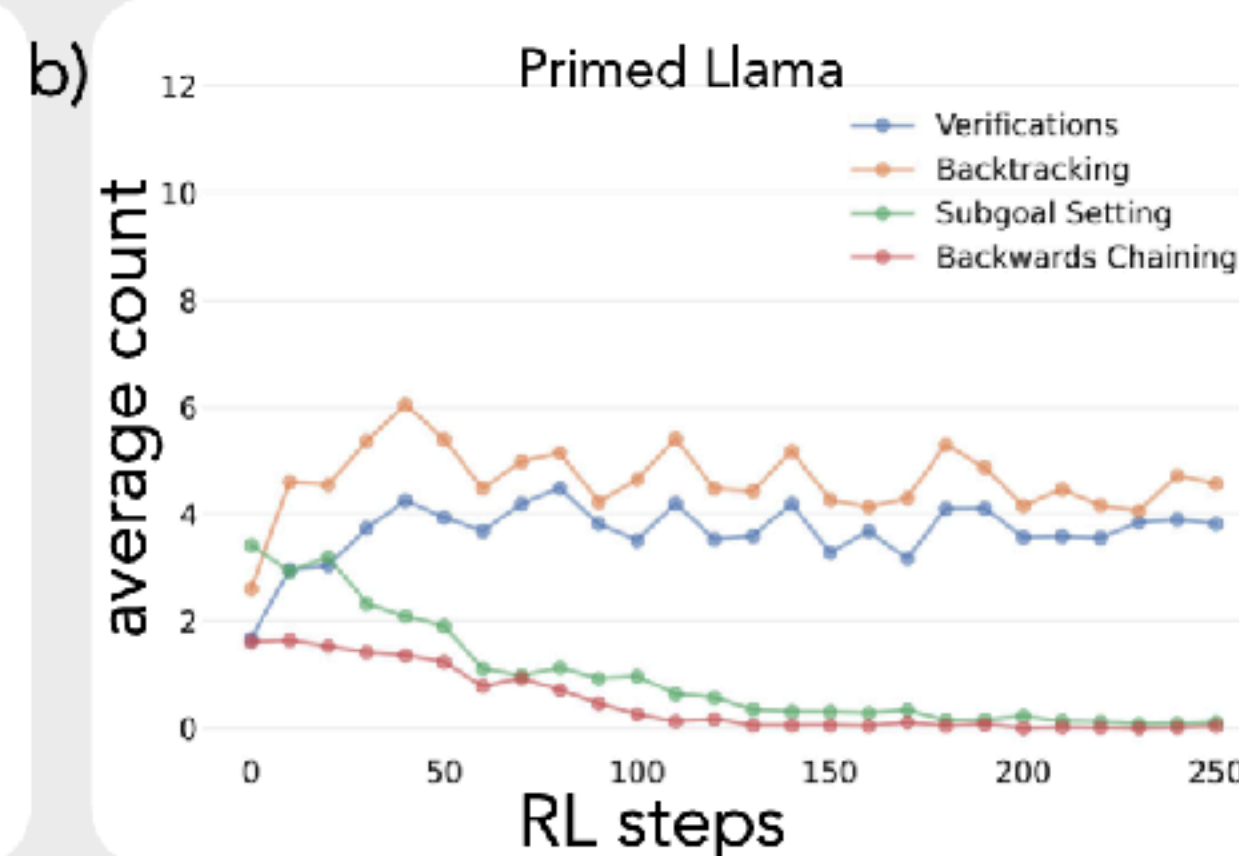
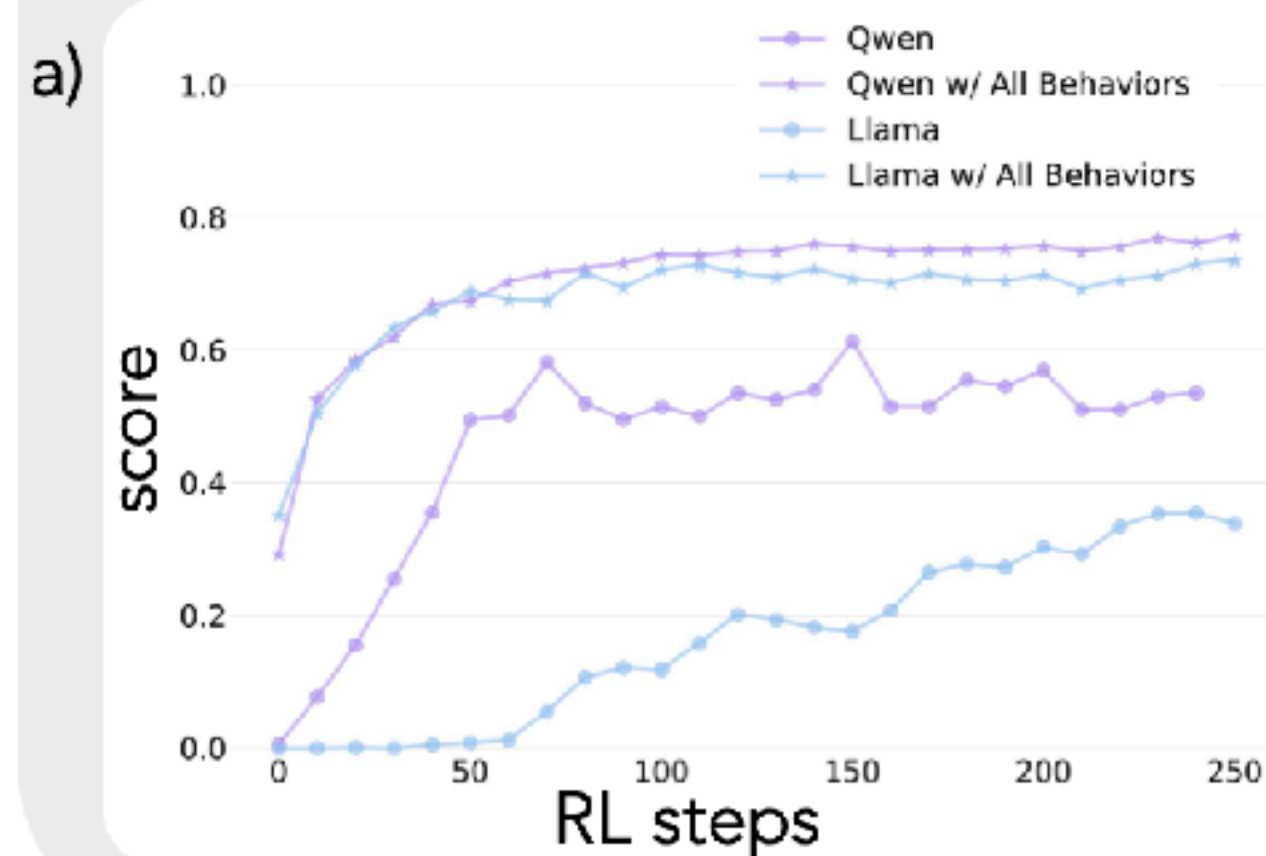
Let's start with the sum of the largest two numbers and then subtract the smallest two: $84 + 83 - 34 - 72$. This gives us $167 - 76$ which equals 91. That's not 39. Let's try another combination: $84 + 83 - 72 - 34$. This gives us $167 - 106$ which equals 61. That's still not 39. Let's try $84 + 72 - 83 - 34$. This gives us $156 - 117$ which equals 39. This is the correct equation.



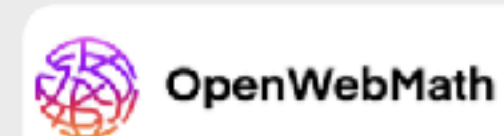
84 is the difference between 108 and 34.
<answer> $(84 - 34) / 108$ </answer>



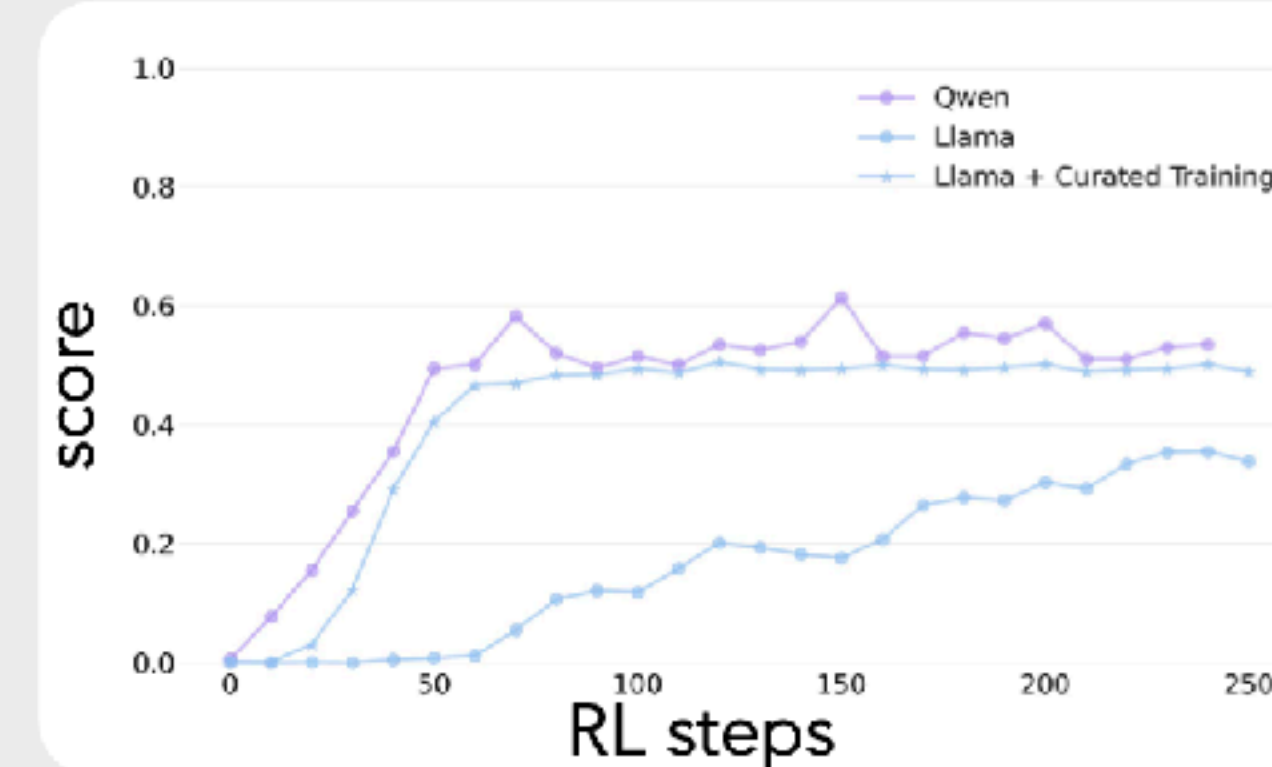
Priming with behaviors reduces performance gap



We can curate a continued pre-training set so that Llama shows similar improvements to Qwen



Filter for behaviors
Reformat as:
Query, Thought,
Answer



Answer Parsing

- As-is, your model could generate the answer anywhere and in any format
- DeepSeek-R1-Zero uses `<answer> ... </answer>` tokens to mark the answer
- In HW2, you'll get a sense for how often answer parsing errors can drive issues

Agents

What's an agent?

- People don't really agree on these criteria, but here are some common ones:
 - Agents are generally able to proactively use tools.
 - Agents generally employ iterative multi-step processes.
 - Agents might have some interaction with the broader world.
- *Examples:*
 - An LM that can browse the web is definitely an agent
 - An LM that can search for files on your computer is definitely an agent
 - A model that can do retrieval based on LMs is probably not an agent
 - An LM with just complex chain-of-thought is not an agent

What's an agent?

- Today's models are usually LLM weights + tools + harness
 - **Tools:** actions that a model can request, like web search, code execution, file editing, APIs
 - **Harness:** software around the weights that executes requests and feeds results back into the LLM context

Web Agents

BrowserGym



Hi! I am your UI assistant. I can perform web tasks for you. What can I help you with?

Create a new hardware asset with a value of "Computer" for field "Model category", a value of "Lenovo ThinkStation S20" for field "Model", a value of "Lenovo" for field "Vendor", a value of "2024-01-01" for field "Installed", and a value of "SN-456-789" for field "Serial number".



Send

servicenow

All

Hardware - N...

Hardware New record

Submit

Display name

* Model category

* Model

Configuration Item

* Quantity

1

General

Financial

Disposal

Depreciation

Contracts

Entitlements

Activities

Asset tag

State

In use

Assigned to

Managed by

Owned by

Parent

Class

Hardware

Serial number

Substate

-- None --

Location

Department

Company

Asset function

-- None --

Assigned

Installed

3

Model category

Computer

Recent selections

Computer

Search

Computer Peripheral

General

Financial

Disposal

Depreciation

Contracts

Entitlements

2

Request line

Invoice number

Cost

\$

1,699.99

Vendor

Lenovd

3

January 2024

S M T W T F S

31 1 2 3 4 5 6

7 8 9 10 11 12 13

14 15 16 17 18 19 20

21 22 23 24 25 26 27

28 29 30 31 1 2 3

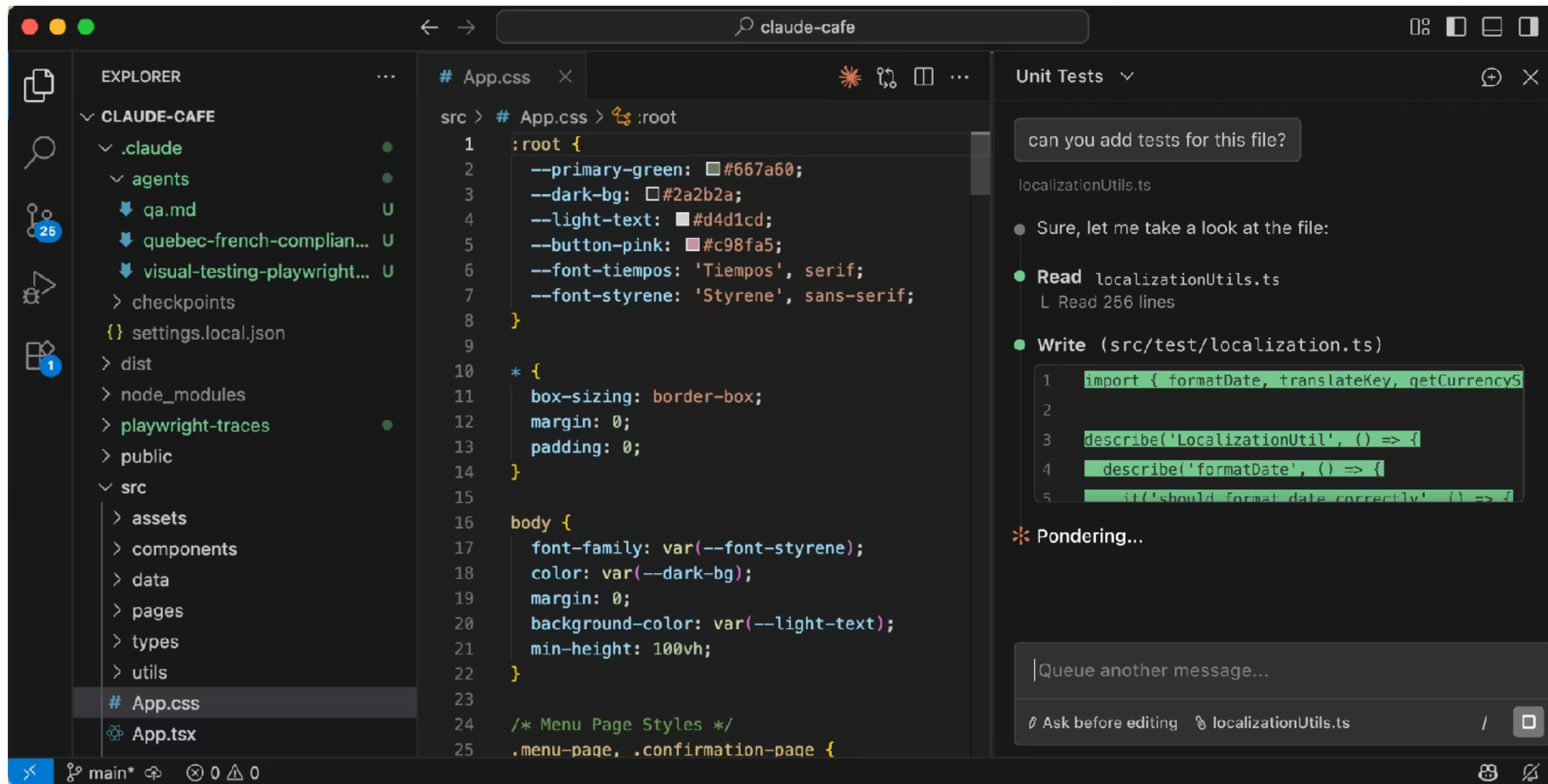
4 5 6 7 8 9 10

Go to Today

Time: 12 13 25

[De Chezelles et al., 2025]

Coding Agents



What makes for a good agent?

Effective tool use

Reasoning and planning abilities

Environment representation

Interaction and communication capabilities

Environment understanding

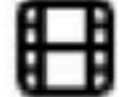
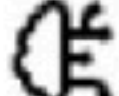
What is a tool?

- In the context of LM-based agents, a **tool** is a **function interface** to some program that runs **externally** to the LM. The LM generates function calls and input arguments to use the tool.
- Examples:
 - Using a calculator program to do math
 - Using web search to find information
 - Using some external model to handle non-text modalities
 - Things like chain-of-thought are not tools! The model itself does them with its own parameters.

Why do we need tools?

- Model weights alone can get us very far - but they're just not good at some things
- **Failure mode 1: knowledge cutoffs**
 - LMs are trained on data up to some date; can't directly answer questions about documents created after that date
- **Failure mode 2: acting in the world**
 - The LMs we've seen so far have not been trained to act in a real-world environment

Types of Tools

Category	Example Tools
 Knowledge access	<code>sql_executor(query: str) -> answer: any</code> <code>search_engine(query: str) -> document: str</code> <code>retriever(query: str) -> document: str</code>
 Computation activities	<code>calculator(formula: str) -> value: int float</code> <code>python_interpreter(program: str) -> result: any</code> <code>worksheet.insert_row(row: list, index: int) -> None</code>
 Interaction w/ the world	<code>get_weather(city_name: str) -> weather: str</code> <code>get_location(ip: str) -> location: str</code> <code>calendar.fetch_events(date: str) -> events: list</code> <code>email.verify(address: str) -> result: bool</code>
 Non-textual modalities	<code>cat_image.delete(image_id: str) -> None</code> <code>spotify.play_music(name: str) -> None</code> <code>visual_qa(query: str, image: Image) -> answer: str</code>
 Special-skilled LMs	<code>QA(question: str) -> answer: str</code> <code>translation(text: str, language: str) -> text: str</code>

Methods of Tool Use

- To use tools effectively, a model must know some important things:
 - When to switch between text generation mode and tool execution mode
 - How to induce tool use
 - Training with tools
 - Test-time prompting

Orchestrators

- The LM simply generates tokens. The **orchestrator** is responsible for parsing tokens, executing tools, and appending results to the LM's context
- Training for tool use just means making the model behave well under this altered way of generating and receiving tokens

```
messages = [...]
while True:
    resp = model(messages, tools=tools)
    if not resp.tool_calls:
        return resp.text

    messages.append(resp.message)
    for call in resp.tool_calls:
        result = execute_tool(
            call.name, call.args)
        messages.append({
            "role": "tool",
            "tool_call_id": call.id,
            "content": result})
```


Example: ReAct

[Yao et al., 2022]

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1a) Standard

Answer: iPod



(1b) CoT (Reason Only)

Thought: Let's think step by step. Apple Remote was originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. So the answer is iPhone, iPad, and iPod Touch.

Answer: iPhone, iPad, iPod Touch



(1c) Act-Only

Act 1: Search[Apple Remote]

Obs 1: The Apple Remote is a remote control ...

Act 2: Search[Front Row]

Obs 2: Could not find [Front Row]. Similar: ...

Act 3: Search[Front Row (software)]

Obs 3: Front Row is a discontinued media center software ...

Act 4: Finish[yes]



(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: Search[Apple Remote]

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

Act 2: Search[Front Row]

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .

Act 3: Search[Front Row (software)]

Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

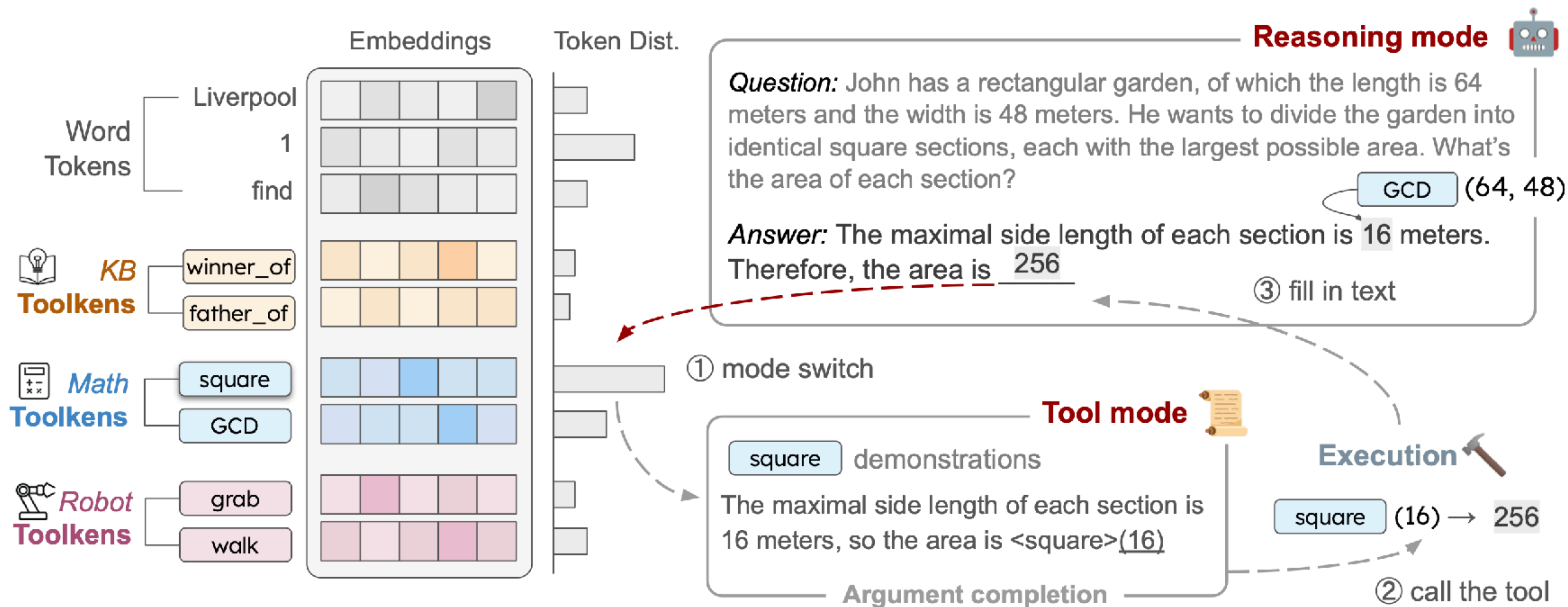
Act 4: Finish[keyboard function keys]



Example: ReAct

- Before ReAct [**Yao et al., 2022**], reasoning/chain-of-thought and tool calls were separate literatures with different methods
- ReAct made tool calling just another form of token generation
- Pretty much every modern agent today uses some version of this approach

Tool Tokens

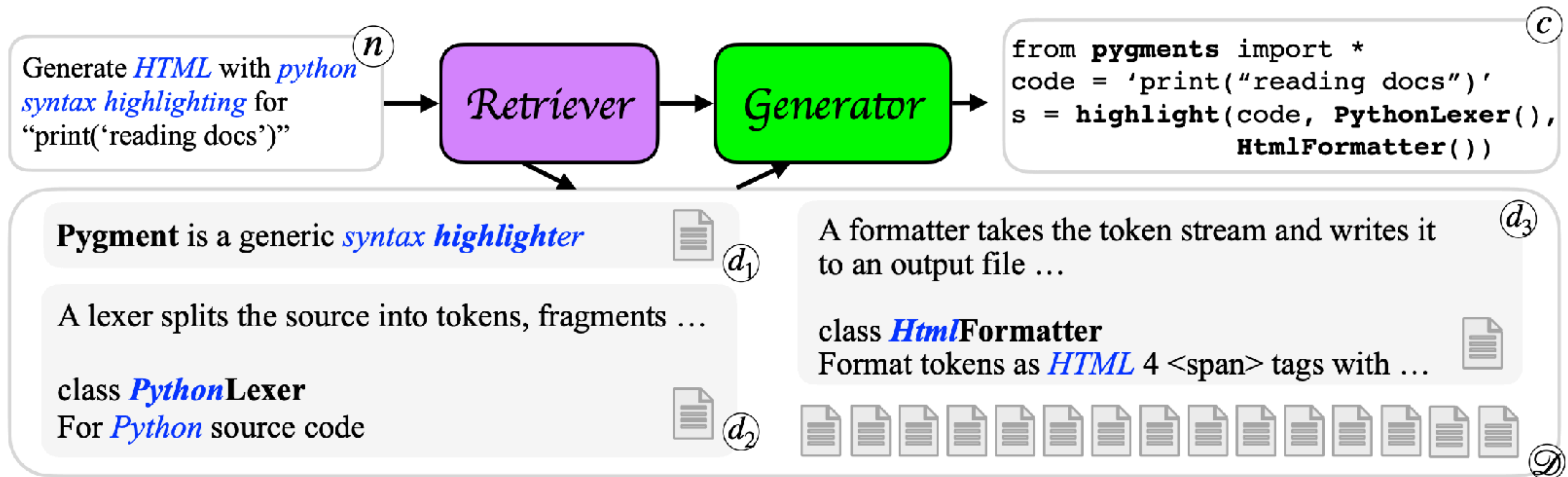


Allegedly the method used by OpenAI models.

Good if you know ahead of time exactly which tools your model will have access to.

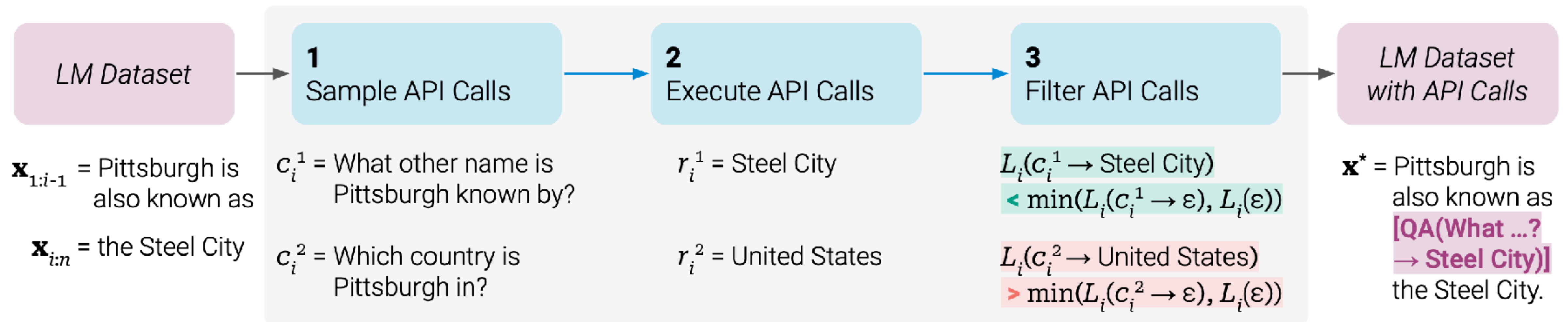
Prompting for Tool Use

- DocPrompting retrieves code library documentation



[Zhou et al., 2023]

Training for Tool Use



- We can filter for tool use cases that successfully reduce the loss on the correct next tokens by at least some threshold, and use those as training examples
 - (Recall the method we mentioned last lecture where we only train on examples where the LM generated the correct answer.)

Training Data for Tool Calling

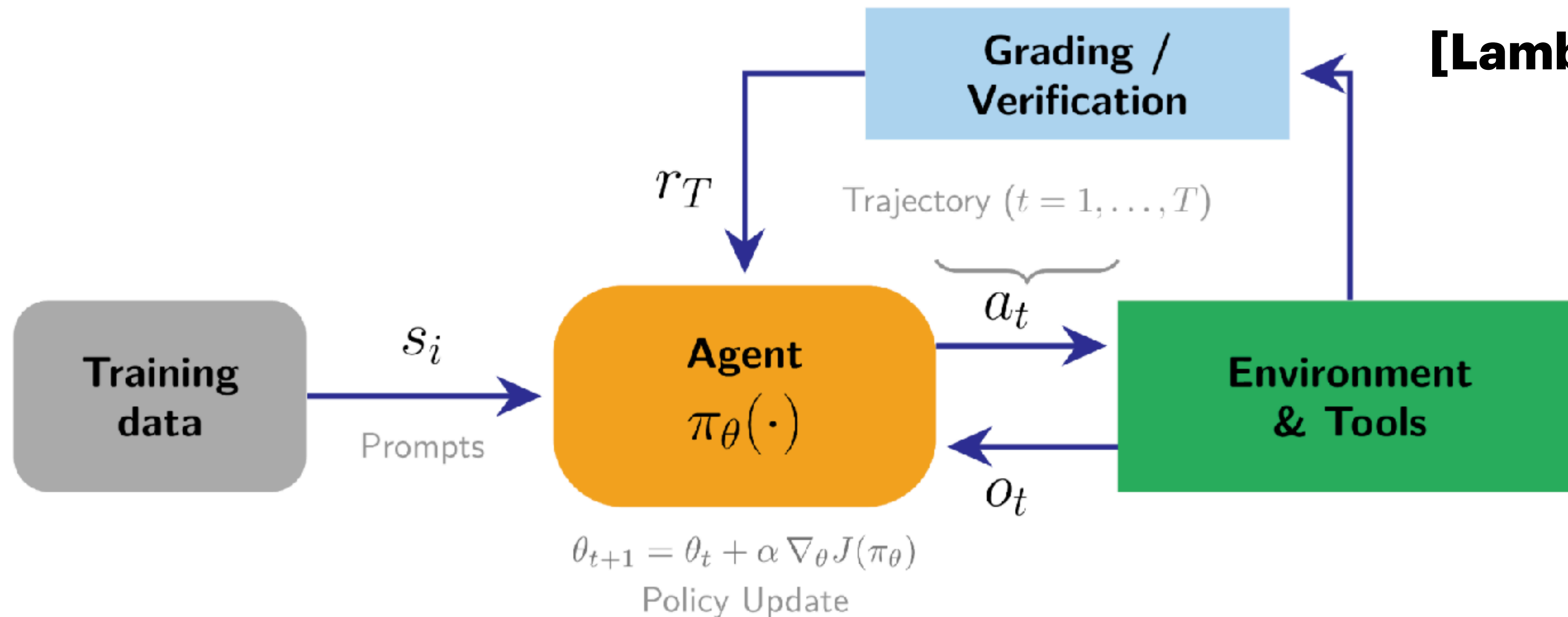
- System prompt declares the available tools
- Model never learns to implement the function itself, nor to connect to anything. Just learns to emit calls in this format and to expect results in-context
- These days, models must generalize to tools they weren't necessarily trained on

```
<system>
You are a function-calling AI model. You are
provided function signatures within <functions>
XML tags. You may call one or more functions.
</system>

<functions>
[
  {
    "name": "search_movies",
    "description": "Search movies by title.",
    "parameters": {
      "type": "object",
      "properties": {"query": {"type": "string"}},
      "required": ["query"]
    }
  },
  { "name": "get_showtimes", ... },
  { "name": "get_movie_details", ... }
]
</functions>
<user> ... </user>
```

Post-training for Tool Use

[Lambert, 2025]



- Works much the same as reinforcement learning without tools; just increases the policy's (agent's) search space

Post-training Data for Tool Calling

- This is much trickier. Agentic RL requires **environments** like containers, file systems, services, verification tests, etc.
- Very easy to curate data that's either trivial or way too hard
 - One tool call, or 30-60? Are all examples equally hard?
 - Domain-specific or multimodal data
 - **Verifiers:** graded checks beyond exact matching - metric thresholds, fuzzy correctness, adversarial corpora
- Environments introduce a ton of new possible failure modes - inactive API keys, CPU resource limits, bandwidth limits, missing data